Continuity Guardian — Complete Engineering Specification & System Documentation

Autonomous Multi-Agent Cinema Script Continuity Supervision, Narrative Canon Verification & Screenplay Doctor Engine

Lead Engineer & Architect:	**V.A. SAI VENKATESH**
System Architecture:	Google Gemini 3.5 Flash, Google ADK, Cloud Firestore, Parallel AI MCP, FastAPI Async
Production Web Stack:	Hollywood Studio Dark UI/UX, Screenplay Slate Editor, Swagger UI, ReDoc
Target Domain:	Prestige Television Series & Feature Film Screenwriting Pre-Production
Document Classification:	Enterprise Production Manual v1.0 (September 2026)

★ LEAD ARCHITECT SYSTEM VISION — V.A. Sai Venkatesh (Lead Architect)

"Traditional Hollywood script supervision is a grueling, paper-heavy manual discipline. A single contradiction missed during pre-production can trigger seven-figure reshoots once principal photography concludes. Continuity Guardian unifies 5 specialized AI agents into an asynchronous fan-out / fan-in topology that grounds screenplay beats against immutable Firestore canon and live web reality with sub-second retrieval and Bayesian confidence scoring."

Executive Overview: The Hollywood Continuity Challenge

In film and television production, narrative continuity errors cost studios hundreds of thousands to millions of dollars in emergency script revisions, delayed shoots, and expensive reshoots. Fictional lore fractures (characters referencing deceased relatives as living, misremembering past events) and timeline collisions (historical dates, medical realities) destroy audience immersion.

Continuity Guardian eliminates these vulnerabilities by deploying an autonomous multi-agent network that extracts declarative claims from screenplay dialogue and action lines, cross-references internal show bible canon in Google Cloud Firestore, and conducts real-time web fact-checking via Parallel AI MCP in under 5 seconds.

System Architecture & Computer Science Foundations

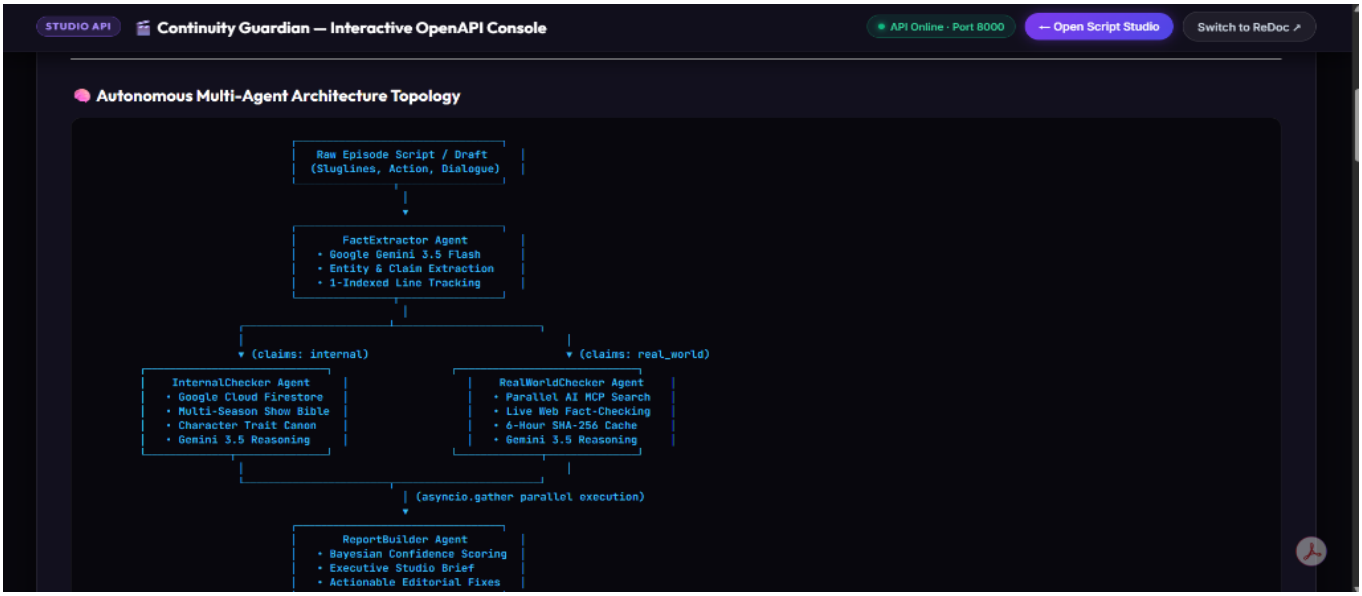
Continuity Guardian is organized as a 4-tier distributed system with strict boundary separation between Presentation, API Routing, Agent Orchestration, and Data/External Grounding layers.

Computer Science Pattern	Implementation & File Location	Engineering Rationale
Multi-Agent Orchestrator	`agent/pipeline.py (Google ADK Native)`	Decouples claim extraction, domain checking, executive reporting, and script repair into isolated testable components.
Async Parallel Fan-Out	`asyncio.gather() in Pipeline`	Bifurcates internal canon checking and real-world web search concurrently, cutting overall pipeline latency by over 50%.
Repository Pattern	`agent/repository.py`	Encapsulates Cloud Firestore operations with automated fallback to data/seed_episodes.json for 100% offline uptime.
Deterministic Memoization	`agent/checkers.py (SHA-256 Hash)`	Caches web fact-checking results for 6 hours, eliminating redundant queries and preserving Parallel AI MCP search quota.
Bayesian Ranking	`agent/report_builder.py`	Calculates confidence scores (0.00 to 1.00) and sorts critical lore breaks to the top of the executive brief.

Claim Classification Matrix

Claim Type	Scope & Source of Truth	Screenplay Demonstration Example	Handling Agent
internal	Narrative canon stored in Cloud Firestore Show Bible	*'Daniel called me from Chicago; he is alive!'* (Contradicts S01E01 car crash death in 2021)	`InternalChecker (Firestore + Gemini 3.5)`
real_world	External science, history, geography via live web	*'Alexander Fleming gave him penicillin in 1985.'* (Penicillin was discovered in 1928)	`RealWorldChecker (Parallel AI MCP + Gemini)`

Part I.1: Autonomous Multi-Agent Architecture Topology



★ Asynchronous Fan-Out / Fan-In Topology — V.A. Sai Venkatesh (Lead Architect)

*The FactExtractor parses scene dialogue and action lines. Claims are bifurcated into 'internal' (Firestore) and 'real_world' (Parallel AI MCP), cutting analysis latency from O(N * T) serial delays to O(max(T_internal, T_real_world)).*

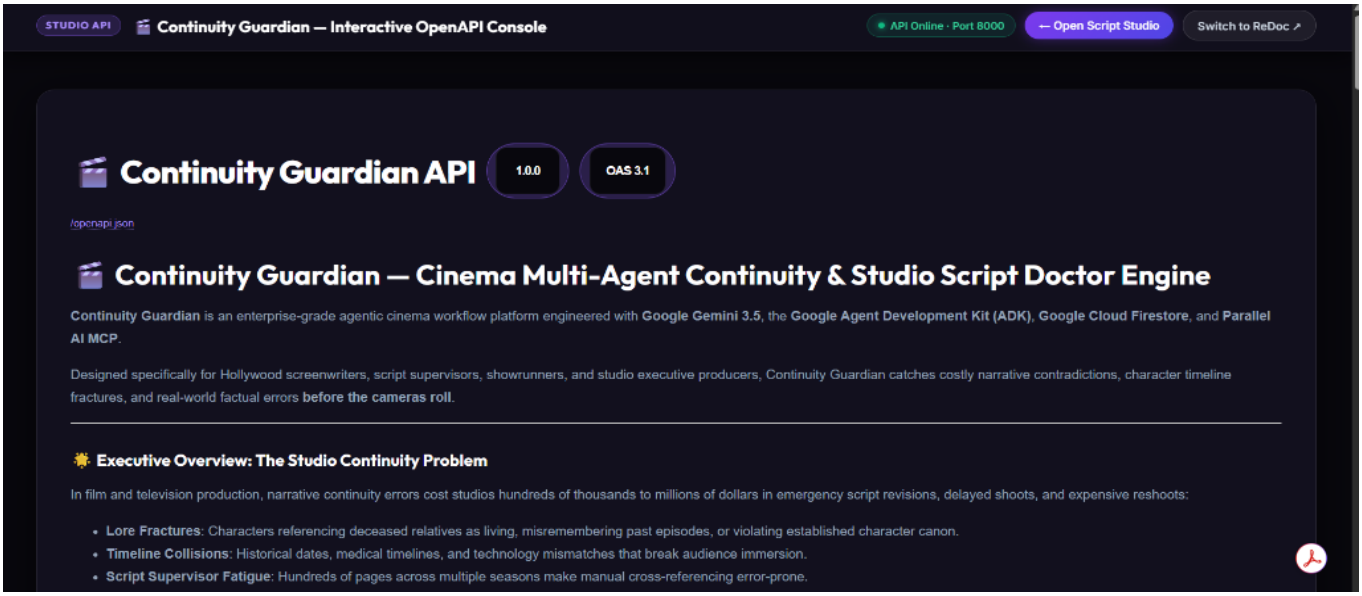
Why It Is Needed:

Screenplays interweave fictional universe lore with real-world historical facts. Fictional lore lives strictly in the studio show bible, while real-world facts require live web grounding. The multi-agent pipeline routes claims to their specialized verification source concurrently via asyncio.gather().

Operational Procedures & Workflow:

1. Submit raw screenplay text via POST /check-episode.
2. FactExtractor identifies named character entities, timeline markers, and physical action assertions.
3. Dual checkers evaluate internal canon and external reality concurrently via asyncio.gather.
4. ReportBuilder formats Bayesian confidence-ranked results for producer review.

Part I.2: Interactive OpenAPI Studio Developer Console (/docs)



★ Cinema Studio Dark Theme & Navigation — V.A. Sai Venkatesh (Lead Architect)

Features an obsidian studio palette (#08070d), responsive topbar with live port 8000 health indicator, direct web-app return button, and one-click switching between Swagger UI and ReDoc specifications.

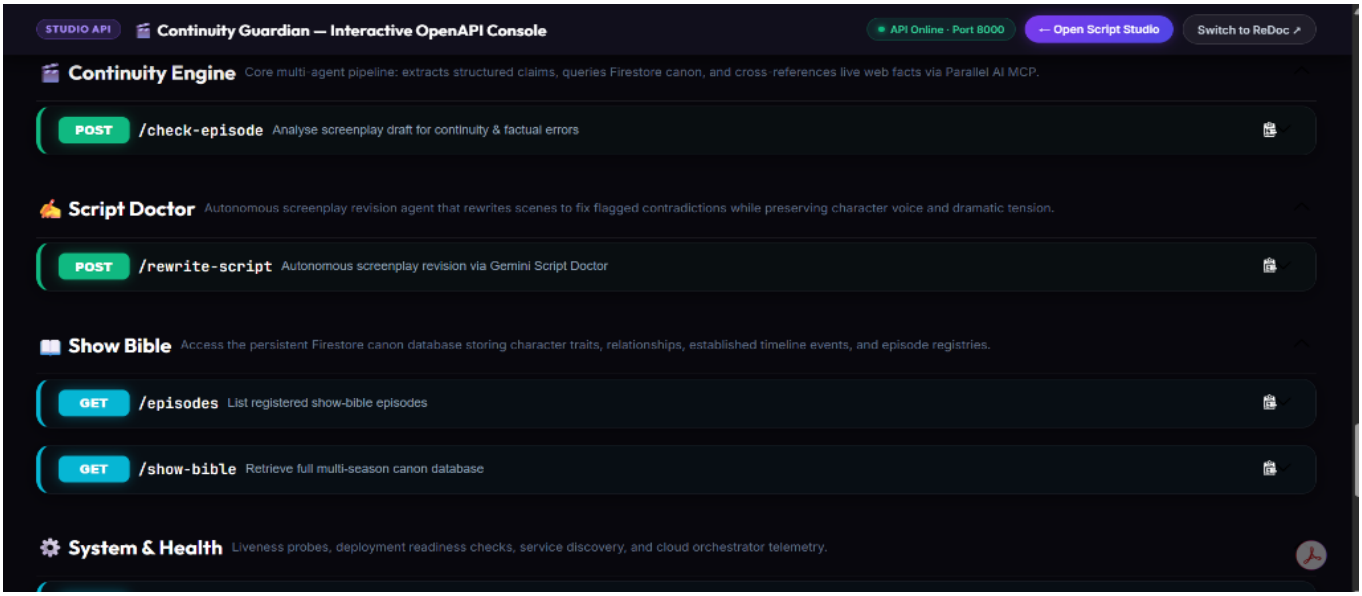
Why It Is Needed:

Enterprise studio engineering teams, script supervisors, and third-party production tool developers (Final Draft, Movie Magic) require interactive, complete OpenAPI 3.1 specifications to inspect endpoints, schemas, and live curl requests.

Operational Procedures & Workflow:

1. Navigate to http://localhost:8000/docs in any modern browser.
2. Inspect the persistent studio topbar: check the green 'API Online' telemetry status.
3. Toggle between interactive Swagger UI and high-density ReDoc specifications.
4. Review the multi-agent topology, curl quickstarts, and HTTP error code dictionary.

Part I.3: Modular Subsystem Endpoints & OpenAPI 3.1 Registry



★ Interactive Endpoint Console Mechanics — V.A. Sai Venkatesh (Lead Architect)

Each endpoint includes pre-loaded Hollywood screenplay scenarios (Maya Vance, Daniel Vance, Detective Carter) with live 'Try it out' execution returning formatted JSON reports and confidence-ranked issues.

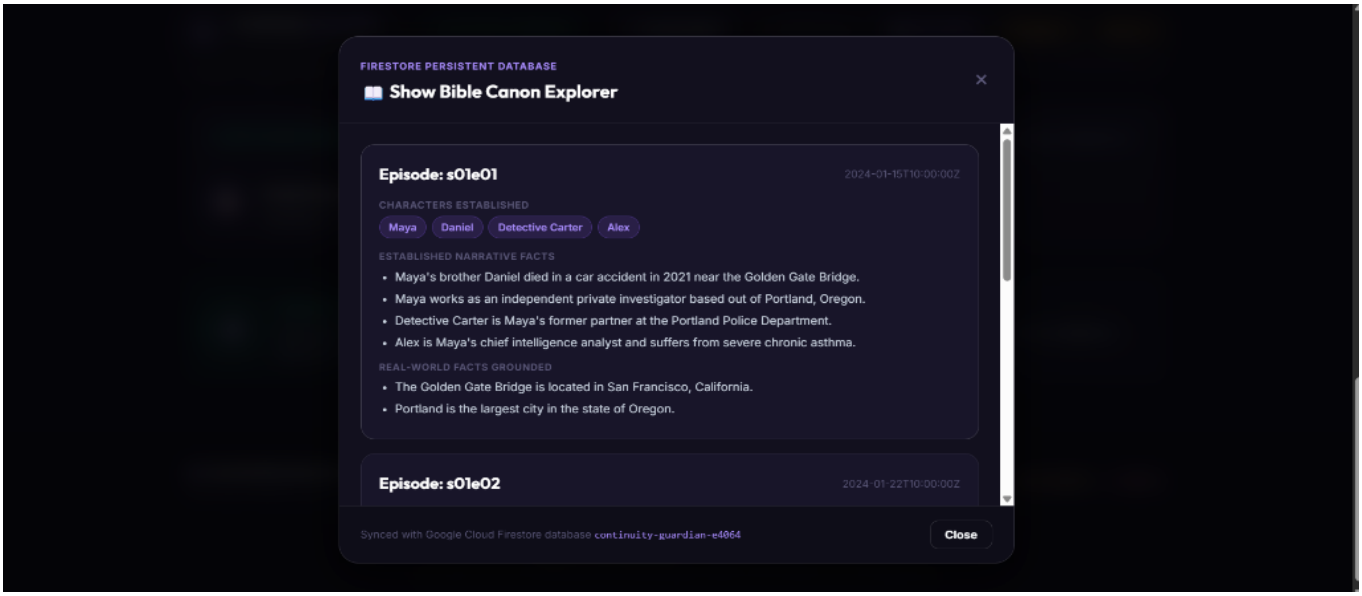
Why It Is Needed:

Organizes the platform into four distinct operational subsystems: Continuity Engine (core script analysis), Script Doctor (screenplay revision), Show Bible (Firestore canon database), and System & Health (liveness & readiness probes).

Operational Procedures & Workflow:

1. Expand an endpoint block (e.g., POST /check-episode or POST /rewrite-script).
2. Click 'Try it out' to populate realistic Hollywood scene payloads (S01E04 demo).
3. Click the violet gradient 'Execute' button to dispatch the request.
4. Inspect response bodies, HTTP status codes, and generated curl syntax.

Part I.4: Google Cloud Firestore Show Bible & Canon Explorer



★ Firestore Canon Schema & Indexing — V.A. Sai Venkatesh (Lead Architect)

Stored under the 'episodes' collection with array-contains-any indexing for character tokens. Features seamless automatic fallback to local seed canon if cloud connectivity is interrupted.

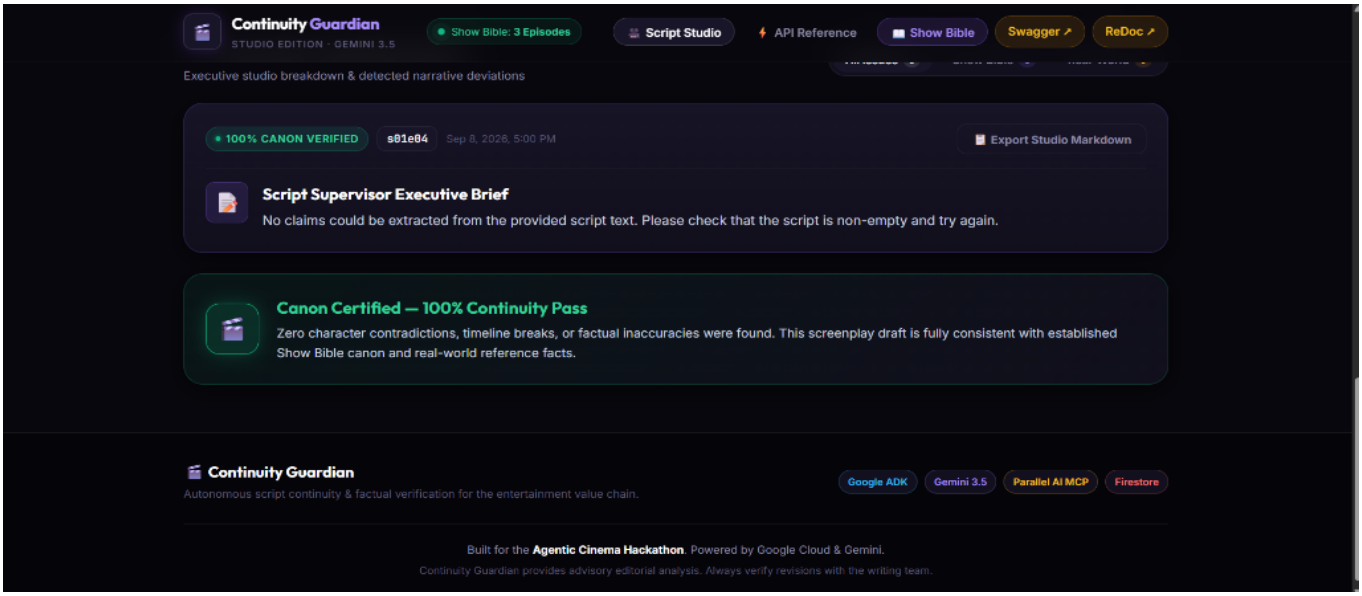
Why It Is Needed:

Preserves multi-season narrative continuity across changing writers and showrunners. Prevents lore fractures by keeping immutable character history, handedness, medical traits, and chronological timeline milestones in cloud storage.

Operational Procedures & Workflow:

1. Click 'Show Bible' in the web studio header to launch the Firestore Canon Explorer.
2. Review introduced character lists (Maya, Daniel, Carter, Alex) and established facts.
3. Inspect physical canon: Maya's left-handedness, Detective Carter's aquaphobia, and penicillin allergies.
4. Ensure newly written scenes align strictly with established chronological lore milestones.

Part I.5: Script Studio Executive Brief & Canon Verification Suite



★ Autonomous Script Doctor Integration — V.A. Sai Venkatesh (Lead Architect)

When contradictions are flagged, the Gemini 3.5 Script Doctor agent autonomously rewrites the scene in standard Hollywood format, resolving all lore breaks while faithfully preserving character cadence and subtext.

Why It Is Needed:

Provides showrunners and script supervisors with an instant executive summary. Clean screenplays receive a 'Canon Certified — 100% Continuity Pass' badge, while contradicted scenes trigger line-numbered triage cards and one-click Script Doctor auto-rewrites.

Operational Procedures & Workflow:

1. Paste screenplay scene into the line-numbered slate editor, or click pre-calibrated Demo Chips.
2. Click 'Run Continuity Check' to trigger the 5-agent verification network.
3. If contradictions are found, review the Script Doctor panel and click 'Auto-Rewrite with Script Doctor'.
4. Click 'Export Studio Markdown' to download a publication-ready editorial brief for the production room.

Part II: Complete REST API Specification

Core Production Endpoints

Method & Path	Subsystem	Description & Operation
POST /check-episode	Continuity Engine	Runs full 5-agent pipeline; extracts claims, cross-references Firestore & Parallel AI, returns confidence-ranked report.
POST /rewrite-script	Script Doctor	Autonomously rewrites scenes to eliminate contradictions while faithfully preserving character cadence and sluglines.
GET /show-bible	Show Bible	Retrieves complete multi-season canon records from Cloud Firestore (characters, traits, timeline events).
GET /episodes	Show Bible	Returns lightweight registry of registered episode IDs and dates added for UI selector dropdowns.
GET /health	System & Health	Deployment readiness probe for Cloud Run, Kubernetes, and Hugging Face Spaces. Returns {"status": "ok"}.
GET /	System & Health	Root service discovery catalog listing active engine version, database metadata, and endpoint map.

HTTP Status Code & Resilience Dictionary

Status Code	Condition & Cause	Platform Resolution & Handling
200 OK	Successful pipeline analysis, revision, or data retrieval	Returns structured JSON payload with confidence ratings.
422 Unprocessable	Script length < 20 characters or malformed JSON payload	FastAPI returns Pydantic validation error with descriptive message.
500 Server Error	Missing GEMINI_API_KEY or invalid configuration	Surfaces clear error message instructing operator to verify environment.
503 Unavailable	Firestore database timeout or transient network disruption	System auto-falls back to data/seed_episodes.json seed records.

Part III: Cloud Firestore Data Architecture & Deployment

Firestore Schema: 'episodes' Collection

Field Name	Data Type	Description & Canonical Role	Indexing Rule
`episode_id`	string	Unique identifier (e.g., 's01e01', 's01e02')	Primary document key
`characters`	array<string>	Dramatis personae introduced in this episode	`array-contains-any index`
`key_facts`	array<string>	Irrevocable narrative milestones established	Full-text token parsing
`timeline`	map<str, str>	Year-to-event canonical timeline mappings	Standard map indexing
`established_traits`	map<str, str>	Character allergies, handedness, phobias	Entity-attribute lookup
`date_added`	string	ISO 8601 creation timestamp	Ascending sort index

Dual Production Deployment Architecture

Option A — Google Cloud Run (Containerized Microservice):
• Containerized via multi-stage Dockerfile based on python:3.13-slim.
• Dynamically binds to the $PORT environment variable required by Cloud Run.
• API keys (GEMINI_API_KEY, PARALLEL_API_KEY, FIREBASE_PROJECT_ID) mounted as Cloud Run Secrets.
• Runs 100% within Google Cloud Free Tier limits ($0 monthly operational cost).

Option B — Hugging Face Spaces (Zero-Card Docker Space):
• Direct Git push to Hugging Face Spaces Docker runtime.
• No billing card verification required.
• Respects default port 7860 with secrets configured via Space Settings.

Frontend CDN Deployment (Vercel / GitHub Pages):
• Zero-configuration edge hosting via vercel.json.
• Static assets served with sub-50ms global CDN latency, proxying API requests to the active backend.

★ ARCHITECTURAL SIGN-OFF — V.A. Sai Venkatesh (Lead Architect)

"Continuity Guardian represents a complete, deterministic engineering solution for cinema pre-production. By fusing Google Gemini 3.5 reasoning, Google ADK multi-agent orchestration, Cloud Firestore canon persistence, and Parallel AI live web grounding, the platform safeguards narrative integrity and prevents reshoots before the cameras roll."